

Building Classification using Machine Learning for Earthquake Risk Management

Team Name: **team_TeYa**

Team Members: Tejas Doypare, Lad Yash Pravin, Yokesh K

October 7, 2024

1 Introduction

Disaster risk management is among the global and utmost priorities, and earthquakes are among the major causes of casualties and economic losses. Building structural integrity is one desirable attribute through which damage may be reduced by a quake. For a capability assessment in resisting forces due to seismic loads is particularly crucial for the case of multi-story structures that exhibit abrupt changes in story stiffness, thereby, becoming susceptible to collapse. To set up seismic vulnerability assessments, quick and accurate identification of structural deficiencies across large inventories of buildings is of paramount importance.

Recent advances are seen with regard to data availability, very recently, with advent and proliferation under routine usage of street-view images by sources such as Google Street View (GSV) towards total visual information for the identification of building types and materials. In other words, the performance of machine learning (ML) models for automatic analysis of such photographs provides an opportunity to make earthquake exposure modelling exceedingly fast and scale up. This project aims to develop an automated ML model using street-view images to classify buildings based on their materials, thus contributing to improved preparedness in earthquake events and mitigation strategies thereof.

2 Notations and Definitions

2.1 Building Classes

In this classification task, investigate five different classes of buildings, each relating to particular materials and structural properties:

2.2 Dataset Notations

- **Train_Data:** Folder containing 2,516 labeled images across five classes (A, B, C, D, S).
- **Test_Data:** Folder containing 478 unlabeled images for which predictions are required.
- **Sample_Submission.csv:** Template for submission with image IDs and placeholder predictions.
- **Classes Mapping:** $A \rightarrow 1, B \rightarrow 2, C \rightarrow 3, D \rightarrow 4, S \rightarrow 5$.

3 Data Preprocessing and Exploration

3.1 Data Loading

The collection of images provided for the model training contains 2,516 such images for training and 478 images for testing, where each training image is given one out of five different class labels. The images have been named in the form of facilitating their respective label extraction without any confusion.

3.2 Class Distribution and Imbalance

There exist classes in the training data that are severely biased against, such as the extremely infrequent class D, with much more representatives present. The distribution of images across the five classes evidenced the necessity of dealing with that imbalance during the training of the model.

Table 1: Building Classes: Material and Properties

Class	Material	Properties
A (1)	Steel	Excessive use of glass or reflective cover, rounded solid surfaces, bare steel skeleton, complex geometry of structural frame, slabs presence, very slender structures.
B (2)	Concrete	Exposed rough concrete, thick and uniform structure skeleton, boxy massing, rough texture, stable opening arrangement.
C (3)	Masonry	Reinforced or unreinforced, exposed masonry walls, low wall ratio, no soft story, presence of much decorative patterns.
D (4)	Wooden	Frame largely exposed, wooden bow windows, a rather more irregular arrangement of openings, pitched roof, with bright color and paint outlines.
S (5)	Steel with Panel	Typical of industrial or retail buildings-developments up to a couple of stories high, rows of openings with wide and sparse lower level openings, non-slanted roof, no ornate texture on the without-covering individual panels.

3.3 Data Preprocessing Techniques

- **Resizing:** The images underwent resizing to an identical size which could be compatible with EfficientNet-B5, which would standardize the dataset.
- **Normalization:** The pixel values were normalized to the range of [0,1] to provide faster and stabler training.
- **Data Augmentation:** The dataset was augmented using random rotation and random flips so as to aid in generalization and help with overfitting.
- **Encoding Labels:** The building class labels were converted from categorical (A, B, C, D, S) to ordinal representation (1, 2, 3, 4, 5) so they can work with classification algorithms.

3.4 Validation Strategy

We used an 80-20 split for training, reserving 20% of the training data as a validation set to monitor the model performance in unseen data during training. Stratified splitting was applied to keep the distribution of different classes the same in both training and validation sets.

4 Software Setup and Experimental Pipeline

4.1 Software Environment

On the other hand, this nAnother set of experiments was conducted on Google Colab with GPU acceleration-gaming on the GPU for the efficient training of deep-learning models. The main libraries and framework used include:

- **PyTorch:** For constructing and training the neural network models.
- **Torchvision:** For image transformation and pre-trained models.
- **NumPy and Pandas:** For data manipulation and analysis.
- **Matplotlib and Seaborn:** For data visualization and plotting the metrics.

4.2 Challenges and Learnings

- **Resource Allocation:** The variance in the availability of GPU had led to occasionally delayed training on Colab. Efficient management of training sessions and saving of checkpoints helped mitigate this.
- **Class Imbalance:** Dealing with the imbalance using data augmentation and weighted loss functions was necessary to enhance model performance across all classes.
- **Overfitting:** Utilizing dropout layers and regularization techniques prevented the model from overfitting, thus ensuring generalizability.
- **Learning Rate Tuning:** Experimenting with various learning rates was necessary for balancing convergence speed and stability.

These challenges highlighted the significance of thorough preprocessing, hyperparameter tuning and regularization to create effective machine learning models.

5 Model Development and Final Approach

5.1 Model Selection: EfficientNet-B5

EfficientNet-B5 has been chosen because of its better performance and efficiency in the task of image classification. The architecture does so well in balancing the model depth and width along with resolution that high performance can be achieved while using fewer parameters compared to different models.

5.2 Model Architecture

- **Pre-trained Backbone:** EfficientNet-B5 has been built and trained upon ImageNet, using transfer learning so as to speed up the training phase and increase performance for small datasets.
- **Custom Classifier:** Replaced the top layer with a fully connected layer whose output was set for five classes, corresponding to class types of buildings.
- **Activation Function:** Softmax activation is used to convert the class probabilities.

5.3 Training Strategy

- **Optimizer:** The Adam optimizer, which offers an adaptive learning rate mechanism, has been one of the best choices as it helps in conceiving convergence faster.
- **Loss Function:** The objective function, cross-entropy loss, has been correctly applied for multi-class classification.
- **Hyperparameters:**
 - Learning Rate: 0.0005
 - Batch Size: 16
 - Number of Epochs: 15
- **Early Stopping:** This enabled the application of early stopping to determine when to halt training, thus preventing overfitting by stopping training once performance no longer improved.

5.4 Implementation Details

```
# Example code snippet for model training
import torch
import torch.nn as nn
from torchvision import models, transforms
from torch.utils.data import DataLoader, random_split

# Define transformations
transform = transforms.Compose([
```

```

transforms.Resize((456, 456)),
transforms.ToTensor(),
transforms.Normalize(mean=[0.485, 0.456, 0.406],
                      std=[0.229, 0.224, 0.225]),
transforms.RandomHorizontalFlip(),
transforms.RandomRotation(10)
])

# Load dataset
dataset = CustomBuildingDataset(train_folder, transform=transform)
train_size = int(0.8 * len(dataset))
val_size = len(dataset) - train_size
train_set, val_set = random_split(dataset, [train_size, val_size])

# Data loaders
train_loader = DataLoader(train_set, batch_size=16, shuffle=True)
val_loader = DataLoader(val_set, batch_size=16, shuffle=False)

# Model initialization
model = models.efficientnet_b5(pretrained=True)
model.classifier = nn.Sequential(
    nn.Linear(model.classifier[1].in_features, 256),
    nn.ReLU(),
    nn.Dropout(0.4),
    nn.Linear(256, 5)
)

# Training loop
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.0005)

for epoch in range(15):
    train_model(...)
    validate_model(...)

```

5.5 Final Model Performance

The final model obtained training set validation accuracy of 51.98% against a validation loss of 1.2897. The classification report in Table 2 gives a detailed analysis for all the classes in regard to precision, recall, and F1-score.

Table 2: Classification Report for Validation Set

Class	Precision	Recall	F1-score
A (Steel)	0.45	0.47	0.46
B (Concrete)	0.31	0.15	0.20
C (Masonry)	0.43	0.60	0.50
D (Wooden)	0.72	0.60	0.65
S (Steel Panel)	0.34	0.38	0.36

6 Evaluation and Results

6.1 Performance Metrics

The core evaluation metric was classification accuracy, defined as the proportion of correctly predicted instances out of all instances. Furthermore, precision, recall, and F1-score were computed for each class to understand the model’s performance better, particularly for the case of class imbalance.

6.2 Confusion Matrix

Confusion matrix reveals the instances where the model correctly and incorrectly classified each class. Excepting some categories where some reasonable progress has been made, the model is doing well in class D (wooden classification); classes B (Concrete) and S (Steel panel), however, were misclassified more, which expresses a huge need to refine bisecting these two categories.

6.3 Error Analysis

The lower performance of the model on Classes B and S indicates that these classes share visual similarities with those of other classes or less-defined features. In a future direction, more sophisticated methods of feature extraction or the inclusion of further data may be attempted to increase the discriminative ability of such classes altogether.

7 Conclusion and Future Work

The project was reported to develop a working machine learning model for building classification using street view images, leading to enhanced earthquake risk management by way of structural vulnerability assessments. The training using EfficientNet-B5 was efficient and yielded fair outcomes, particularly with classes where distinctive features were presented.

Key Learnings:

- In addition to high-quality work on the factors moderating true or real effects on adapting dynamic stochastic programs to real applications, achievements must also be aimed at fair model performances for all the classes or categories involved.
- A quite brilliant use of transfer-learning techniques and possibly representation through pre-trained networks to boost performance in a limited-data scenario.
- It is always necessary to have comprehensive data augmentation for increased model generalization.

Future Directions:

- Use some ensemble methods to combine models for better accuracy.
- Study advanced augmentation methods and synthetic data generation to go a step further in avoiding class imbalance.
- More sophisticated feature-extraction techniques to separate visually similar classes.
- More sophisticated validation methods-such as k-fold cross-validation-for more dependable performance estimates.

8 References

- Tan, M., & Le, Q. V. (2019). *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. In International Conference on Machine Learning (pp. 6105-6114).
- Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). *ImageNet Classification with Deep Convolutional Neural Networks*. In Advances in Neural Information Processing Systems (pp. 1097-1105).
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep Residual Learning for Image Recognition*. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (pp. 770-778).